

SymPy Bug Report

1 Bug 22: solveset returns 1 for $\text{acos}(x) = 2\pi$ outside the principal acos range

1.1 Minimal Reproducer

```
from sympy import Eq, S, N, acos, pi, solveset, symbols

x = symbols("x")
eq = Eq(acos(x), 2*pi)
sol = solveset(eq, x, domain=S.Complexes)

print("solution =", sol)
for s in sol:
    print("residual at", s, "=", N(acos(s) - 2*pi, 80))
```

1.2 Observed Behavior

```
solution = {1}
residual at 1 =
-6.2831853071795864769252867665590057683943387987502116419498891846156328125724180
```

SymPy returns the candidate $x = 1$. Substituting that value into the original equation gives

$$\text{acos}(1) - 2\pi = -2\pi,$$

so the returned value does not satisfy the equation.

1.3 Expected Behavior

The correct solution set over $S.Complexes$ is $\emptyset$. The principal value of $\text{acos}(1)$ is 0, not 2π , and the principal branch of acos does not attain the real value 2π .

1.4 Mathematical Explanation

The equation being solved is

$$\text{acos}(x) = 2\pi.$$

It is tempting to apply $\cos$ to both sides and infer

$$x = \cos(2\pi) = 1.$$

That inference is invalid for the principal inverse cosine. The identity $\cos(\arccos(x)) = x$ holds on the appropriate domain, but $\arccos(\cos(y))$ does not equal y for arbitrary y . Instead, $\arccos$ returns a principal value. In particular,

$$\arccos(1) = 0,$$

so $x = 1$ satisfies $\arccos(x) = 0$, not $\arccos(x) = 2\pi$. The reported answer is therefore an extraneous solution introduced by treating a principal-branch inverse as though it were a globally valid inverse.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant solver code in `sympy/solvers/solveset.py`, specifically `_solveset` and `_invert_complex`. The traced call path starts from the complex-domain call to solve $\arccos(x) = 2\pi$. The equation is normalized to $\arccos(x) - 2\pi$. Because this is not classified as a `TrigonometricFunction` equation, `_solveset` takes the generic inversion path and calls the inverter on the normalized equation. The complex inverter removes the additive -2π , obtaining $\arccos(x) = 2\pi$, and then uses the generic inverse-function branch.

The key rule in `_invert_complex` is:

```
if hasattr(f, 'inverse') and f.inverse() is not None and \
    not isinstance(f, TrigonometricFunction) and \
    not isinstance(f, HyperbolicFunction) and \
    not isinstance(f, exp):
    ...
    return _invert_complex(f.args[0],
                           imageset(Lambda(n, f.inverse()(n)), g_ys), symbol)
```

For $f = \arccos(x)$, the inverse method returns $\cos$. That inverse is defined in `sympy/functions/elementary/trigonometric.py`. With target set $\{2\pi\}$, this branch constructs an image set equivalent to $\{\cos(2\pi)\}$, which evaluates to $\{1\}$. The diagnosis further notes that `domain_check` only checks that the original expression is defined at $x = 1$; it does not verify that the residual is zero. Thus the false finite candidate survives even though $\arccos(1) - 2\pi = -2\pi$.

A plausible fix direction supported by the diagnosis is to avoid using this generic inverse branch unconditionally for principal inverse functions. The solver could handle inverse trigonometric functions with explicit principal-range conditions, or it could filter finite inverted candidates by checking that they satisfy the original equation.

1.6 Additional Failing Instances

The same mechanism appears for other real targets outside the principal range of $\arccos$. Applying $\cos$ to those targets produces 1 or -1 , but the principal values $\arccos(1) = 0$ and $\arccos(-1) = \pi$ do not equal the requested targets. The independent verification records the following parametrized cases.

Input / Expression	SymPy behavior	Expected behavior
$\text{acos}(x) = 2\pi$	returns a set containing 1	$\emptyset$
$\text{acos}(x) = -\pi$	returns a set containing -1	$\emptyset$
$\text{acos}(x) = 3\pi$	returns a set containing -1	$\emptyset$
$\text{acos}(x) = 4\pi$	returns a set containing 1	$\emptyset$
$\text{acos}(x) = -2\pi$	returns a set containing 1	$\emptyset$
$\text{acos}(x) = 5\pi$	returns a set containing -1	$\emptyset$

The expected behavior is the same across the family because each target is a real value outside the principal real range of `acos`, while the returned candidate is only a solution of the periodic cosine equation obtained after an invalid global inversion step.

1.7 Related Issues Assessment

The deduplication evidence found related background on principal branches of inverse trigonometric functions and the SymPy `solveset` contract, but no public report of this exact reproducer or output. The verdict is therefore a new instance of a known failure mode: the broad issue of respecting principal branches is known, but this exact `acos` case with the false result `{1}` appears previously unreported and remains individually actionable as a focused issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Eq, S, acos, pi, solveset, symbols, simplify

x = symbols("x")

CASES = [
    (2*pi, 1),
    (-pi, -1),
    (3*pi, -1),
    (4*pi, 1),
    (-2*pi, 1),
    (5*pi, -1),
]

@pytest.mark.parametrize("target,bad", CASES)
def test_solveset_acos_rejects_values_outside_principal_range(target, bad):
    sol = solveset(Eq(acos(x), target), x, domain=S.Complexes)

    assert sol == S.EmptySet
    assert simplify(acos(bad) - target) != 0
```